



实验安排

主要内容

1 DE1-SoC\Quartus\SignalTap

2 ALU 设计

3 单周期 CPU 设计



1.DE1-SoC\Quartus\ SignalTap

实验内容一： My_First_Fpga

实验内容二： SignalTap 的使用



• 实验内容一： My_First_Fpga

实验目的：

学习开发板 **DE1-SoC**、开发工具 **Quartus II 14.0** 和调试工具 **SignalTap** 的使用方法。

实验内容：

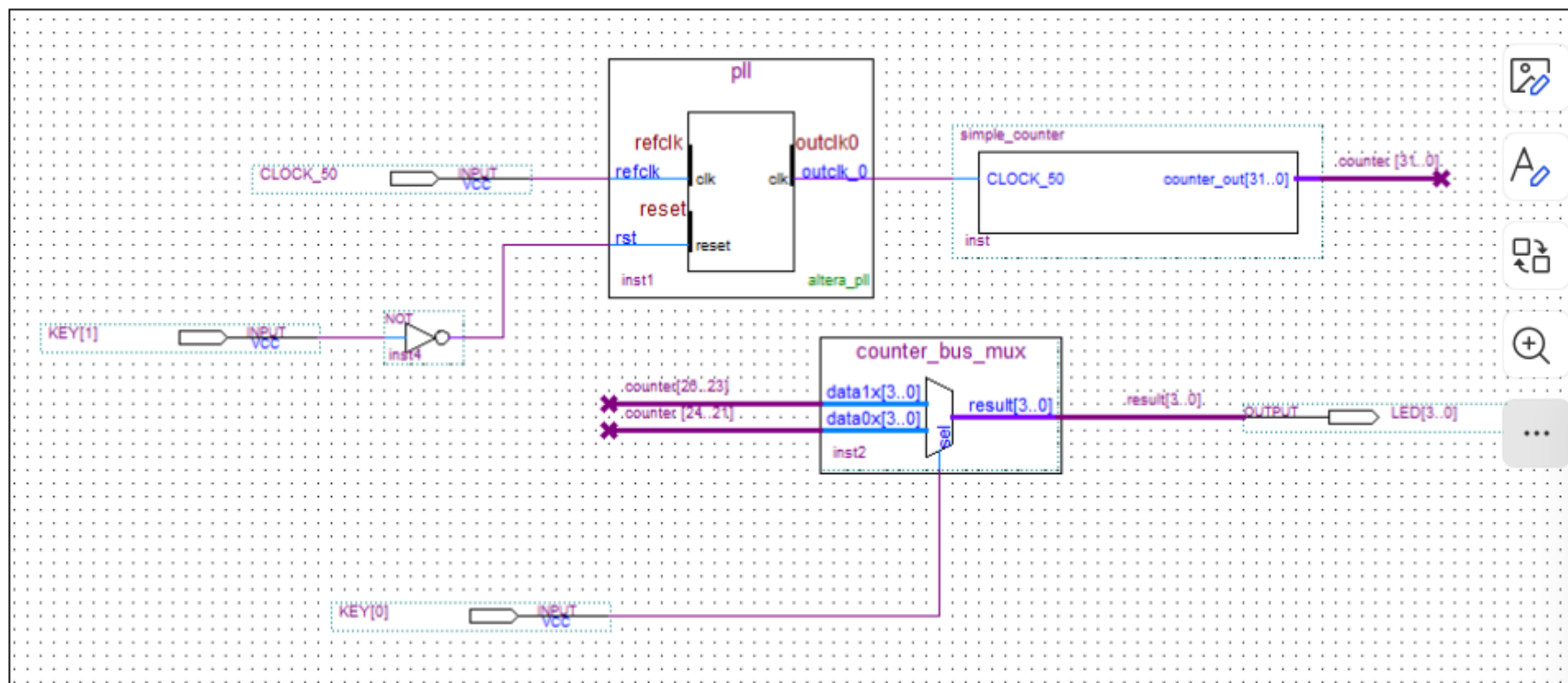
通过输入键控制开发板上 **LED** 的闪烁速度。

实验要求：

完成设计并下载至 **FPGA** 开发板进行验证。



实验内容一： My_First_Fpga



说明：LED 的设计，需要设计一个简单的 32 位计数器，用 Verilog HDL 代码实现。添加一个锁相环（PLL）作为时钟源，驱动自己定义的计数器。

现象：观察 4 个 LED 灯，以二进制计算模式显示计数。按下 KEY[0]，灯显示加快了，此时多路选择器选择了低位的 bits[24..21]



实验准备

- 计算机进入 win7 系统 (第一个选项)、连接开发板 DE1-SOC 电源和 USB 线、打开 Quartus 14.0 。
- Quartus 14.0

Altera Quartus II 是一种可编程逻辑的设计环境，设计能力强大，接口直观易用。

- DE1-SOC

开发板的处理器芯片是 Altera 的 FPGA 芯片， Cyclone V SoC 5CSEMA5F31C6 。整块板子被称为 DE1-SOC 。



- 调试工具 **SignalTap**

功能强大，FPGA 片上 debug 工具软件，它集成在 Quartus II 中；

由逻辑电路组成，而不是仿真，不要把 Signaltap 和 Modelsim、multisim 混为一谈。



实验说明

1. 使用 Quartus II 时，应**先创建工程**。

➤ File > New Project Wizard，创建过程中需要选择 top_level designe Entity, 其名称应与工程同名。

➤ 创建工程时选用器件 **5CSEMA5F31C6**；

2. 要**设置顶层文件** top_level Entity, 相当于 main 函数。

➤ 原理图文件 *.bdf 和 Verilog 文件 *.v 等价，二者都可以成为 top_level。

➤ 设置 top_level 可以在保存文件时，也可以在 Hierarchy 标签中选中该文件，然后右键设置为 top_level。



实验说明

3.qsf 文件的导入

- 引脚文件是 *.qsf 文件，工具 **SystemBuilder** 可以生成每个资源所连的引脚，包括拨码开关、LED、7 段数码管、按键等。
- 导入方法为 Assignments>Import Assignments。

4. 在 *.bdf 文件中添加 Symbol

- 用右键 ->insert->symbol。自己创建的 symbol 不在默认目录，添加时需选择工程所在目录；



实验说明

5. 实验中使用的 **PLL** 是锁相环模块；是 Megafunction 中预定义好的功能模块，

- Megacore 是功能更复杂的模块，需要 license；
- 文档中的 13.1 不同，**PLL** 不在 **Edit** 下，而是整合在了 Tools->IP catalog->Basic functions->Clocks; PLLs and Reset->PLL->Altera PLL

6. 分配管脚前应运行 processing>start>start analysis &Elaboration, 否则找不到管脚。



实验说明

- 生成的**烧写**文件 *.sof 在工程路径的 **output_files** 下。
- 烧写 **failed** 很可能是**器件选择**错误。
- **MUX 多路选择器**在导入时，勾选生成 **Quartus II symbol file**
- 实验现象：
 - 观察 4 个 LED 灯，以二进制计算模式显示计数。
(simple_counter bits[26..23] 驱动)
 - 按下 **KEY[0]**，观察灯显示加快了，因为多路选择器选择了 bits[24..21]



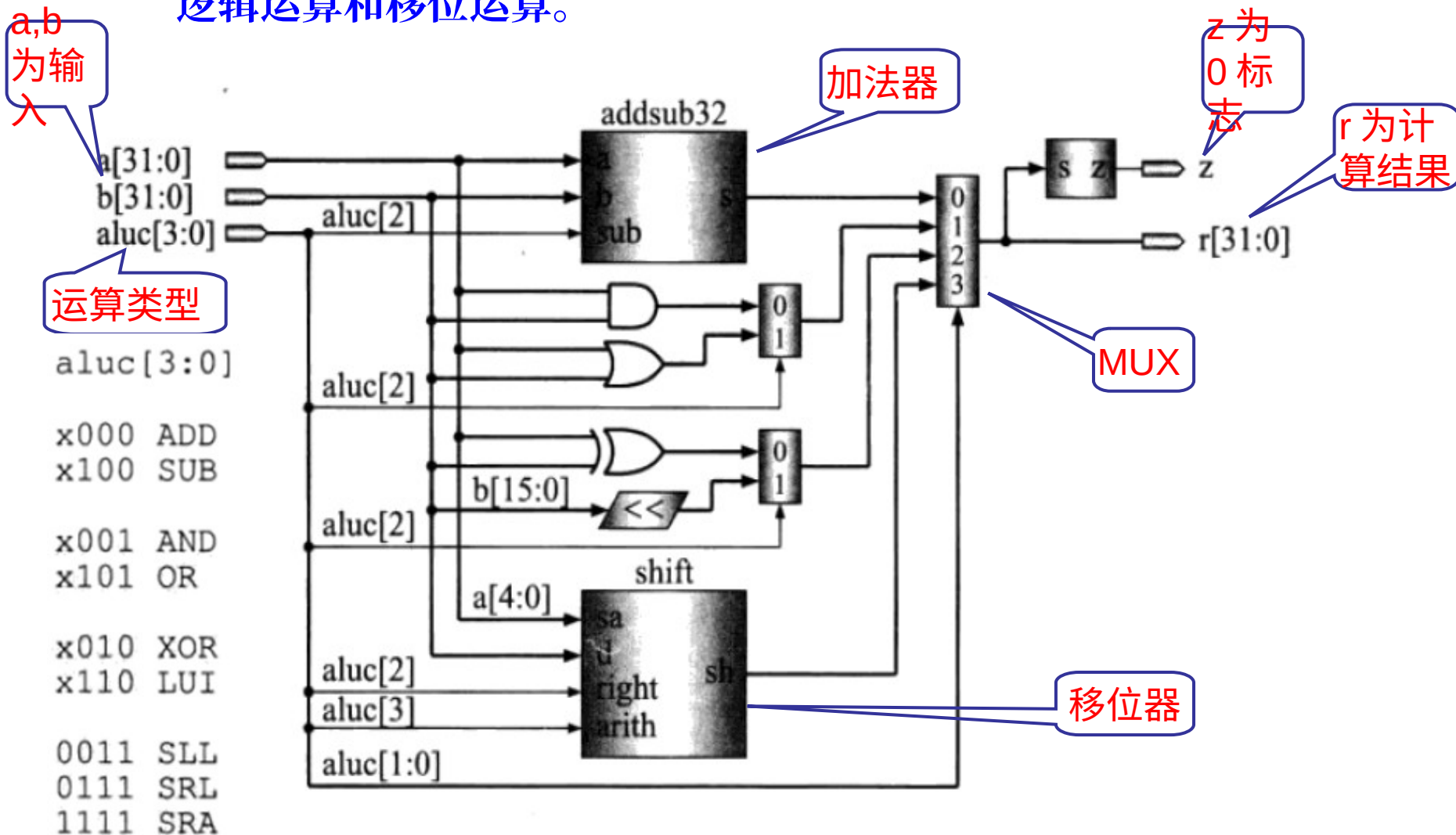
实验内容二： SignalTap 的使用

- 根据附录 PPT，学习调试方法。
- 在 SignalTap 试验中用到 DE1_SOC.qsf 文件，这个文件可用光盘上的工具 DE1SoC_SystemBuilder 生成。
- 使用 signaltap 使用指南 .pdf 也可；网上材料也可参考，只要会使用 signaltap 即可。
- License 问题：运行 quartus 后提示未能授权怎么办？请进入该软件的 tools-license setup，在 license file 框中填写：1800@172.16.121.180 即可成功授权。



2. ALU 的设计

- 实验目的：设计一个 32 位 ALU，能够实现基本的算术运算、逻辑运算和移位运算。





- 所有模块代码均已给出，在目录 **DE1_SOC_test_sy2** 下，理解其代码的**逻辑层次和模块功能**。 **DE1_SOC_golden_top.v** 关键代码：

```
if (!KEY[0])
begin
    pattern_a <= 32'h000_000a;
    pattern_b <= 32'h000_000b;
end
else
begin
    if (SW[4]) // enable patten change
    begin
        pattern_a <= pattern_a + 32'h1;
        pattern_b <= pattern_b + 32'h3;
    end
    else // patttern keep the same value
    begin
        pattern_a <= pattern_a;
        pattern_b <= pattern_b;
    end
end
end
```



- KEY 按下为 0，不按为 1。
- KEY[0] 按下为 a 和 b 赋初值，之后可用于计算；
- SW[4] 拨到 ON 后 a 自加 1，b 自加 3；
- 运算方式 aluc[3:0] 由 SW[3:0] 控制，共支持 9 种运算。拨动开关后运算方式会改变，如下图。

```
aluc[3:0]      if(!KEY[0])
                begin
                    pattern_a <= 32'h000_000a;
                    pattern_b <= 32'h000_000b;
x000 ADD      end
x100 SUB
                else
x001 AND      begin
x101 OR        if(SW[4]) // enable patten change
                begin
x010 XOR        pattern_a <= pattern_a + 32'h1;
x110 LUI        pattern_b <= pattern_b + 32'h3;
                end
0011 SLL      end
0111 SRL      else // patttern keep the same value
1111 SRA      begin
                pattern_a <= pattern_a;
                pattern_b <= pattern_b;
                end
            end
        end
```



- alu.v 如下。

```
module alu (a,b,aluc,r,z);  
    input [31:0] a,b;  
    input [3:0] aluc;  
    output [31:0] r;  
    output z;  
    wire [31:0] d_and = a & b;  
    wire [31:0] d_or = a | b;  
    wire [31:0] d_xor = a ^ b;  
    wire [31:0] d_lui = {b[15:0],16'h0};  
    wire [31:0] d_and_or = aluc[2]? d_or : d_and;  
    wire [31:0] d_xor_lui = aluc[2]? d_lui : d_xor;  
    wire [31:0] d_as,d_sh;  
    addsub32 as32 (a,b,aluc[2],d_as);  
    shift shifter (b,a[4:0],aluc[2],aluc[3],d_sh);  
    mux4x32 select (d_as,d_and_or,d_xor_lui,d_sh,aluc[1:0],r);  
    assign z = ~|r;  
endmodule
```



- addsub32.v 如下，可见它包含一个 32 位超前进位加法器 **cla32**

```
module addsub32 (a, b, sub, s);  
    input [31:0] a, b;  
    input      sub;  
    output [31:0] s;  
    cla32 as32 (a, b^{32{sub}}, sub, s);  
endmodule
```

- 查看 *.v 文件又可见 32 位超前进位加法器是由 16 位拼接，16 位由 8 位拼接…，以此类推。
- 最终可以发现 cla_2.v 包含一个 g_p 的实例。

- add.v
- addsub32.v
- alu.v
- alu_main.v
- cla_2.v
- cla_4.v
- cla_8.v
- cla_16.v
- cla_32.v
- cla32.v
- g_p.v
- mux2x5.v
- mux2x32.v
- mux4x32.v
- shift.v



- **g_p.v** 是进位的生成函数和传递函数，在超前进位加法器中使用。

```
module g_p (g,p,c_in,g_out,p_out,c_out);  
    input [1:0] g,p;  
    input c_in;  
    output g_out,p_out,c_out;  
    assign g_out = g[1] | p[1] & g[0];  
    assign p_out = p[1] & p[0];  
    assign c_out = g[0] | p[0] & c_in;  
endmodule
```

- add.v
- addsub32.v
- alu.v
- alu_main.v
- cla_2.v
- cla_4.v
- cla_8.v
- cla_16.v
- cla_32.v
- cla32.v
- g_p.v
- mux2x5.v
- mux2x32.v
- mux4x32.v
- shift.v



```
module mux4x32 (a0,a1,a2,a3,s,y);  
    input [31:0] a0,a1,a2,a3;  
    input [1:0] s;  
    output [31:0] y;  
  
    function [31:0] select;  
        input [31:0] a0,a1,a2,a3;  
        input [1:0] s;  
        case (s)  
            2'b00: select = a0;  
            2'b01: select = a1;  
            2'b10: select = a2;  
            2'b11: select = a3;  
        endcase  
    endfunction  
    assign y = select(a0,a1,a2,a3,s);  
endmodule
```

32 位 4 选 1 多
路选择器代码是
mux4x32.v

- ☐ add.v
- ☐ addsub32.v
- ☐ alu.v
- ☐ alu_main.v
- ☐ cla_2.v
- ☐ cla_4.v
- ☐ cla_8.v
- ☐ cla_16.v
- ☐ cla_32.v
- ☐ cla32.v
- ☐ g_p.v
- ☐ mux2x5.v
- ☐ mux2x32.v
- ☐ mux4x32.v
- ☐ shift.v



2. ALU 的设计

```
module shift (d,sa,right,arith,sh);
```

```
input [31:0] d;
```

```
input [4:0] sa;
```

```
input    right,arith;
```

```
output [31:0] sh;
```

```
reg [31:0] sh;
```

```
always @* begin
```

```
    if (!right) begin
```

```
        sh = d << sa;
```

```
    end else if (!arith) begin
```

```
        sh = d >> sa;
```

```
    end else begin
```

```
        sh = $signed(d) >>> sa;
```

```
    end
```

```
end
```

```
endmodule
```

算术移位逻辑
移位模块
shift.v

-  add.v
-  addsub32.v
-  alu.v
-  alu_main.v
-  cla_2.v
-  cla_4.v
-  cla_8.v
-  cla_16.v
-  cla_32.v
-  cla32.v
-  g_p.v
-  mux2x5.v
-  mux2x32.v
-  mux4x32.v
-  shift.v

>>> 带符号右移
>> 无符号

本页放弃



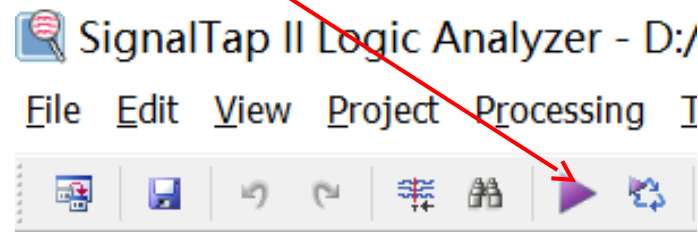
验证设计的正确性

- 用 **signal tap** 工具验证，实验方法看实验一。
- **Verilog** 记不清了可参考百度文库 **<Verilog 教程>**。
<https://wenku.baidu.com/view/bd2b51a94a7302768f993984.html>



第 3~5 次实验

- 用三次实验时间理解单周期 CPU 的设计，CPU 的 verilog 代码在 sccpu 路径下。
- 主要模块有：ALU、控制器、寄存器堆、数据存储器、指令存储器。
- 一段程序已经存储在指令存储器中，数据存储器中也存有少量数据。实验要用设计出的 CPU 运行这段程序。
- 整个工程可以直接编译运行。
- 调试使用 signaltap, 工程中的 stp1.stp 也是可以直接使用的。双击打开 stp1.stp，完全编译 (若用最右边按钮快速编译有时不通过，原因不明。约 5 分钟)，编译完成后下载 sof 文件到开发板。
- 之后按开始调试按钮；
然后 KEY[0] 键即可抓取信号。

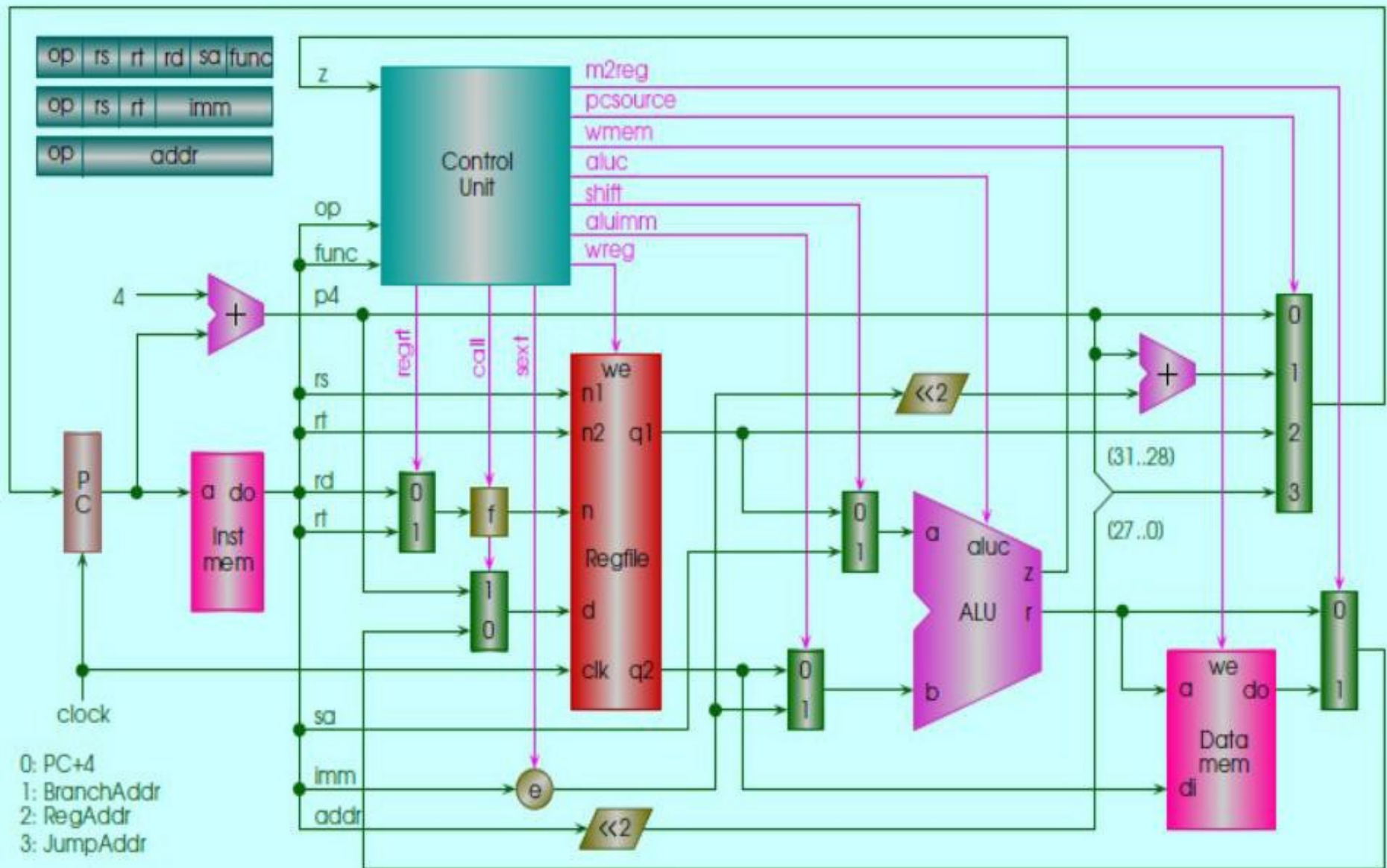




第 3~5 次实验

- 采集到的信号如下图，从图中可见 PC 从 0 到 4 到 8 到 C；其它信号也在随时钟改变，信号取值的变化是指令执行的结果，指令在 scinstmem.v 中，将二者令结合才能理解其含义。

log: Trig @ 2017/05/03 18:57:20 (0:0:6)			click to insert time bar									
Type	Alias	Name	-1	0	1	2	3	4				
		KEY[0]										
		⊕ sccpu_dataflow:s alu[31..0]		00000000h	X	00000050h	X	00000004h	X	00000000h		
		sccpu_dataflow:s clock										
		⊕ sccpu_dataflow:s data[31..0]					00000000h					
		⊕ sccpu_dataflow:s inst[31..0]		3C010000h	X	34240050h	X	20050004h	X	0C000018h	X	00004020h
		⊕ ...pu_dataflow:s mem[31..0]		00000000h	X	000000A3h	X			00000000h		
		⊕ sccpu_dataflow:s pc[31..0]		00000000h	X	00000004h	X	00000008h	X	0000000Ch	X	00000060h
		sccpu_dataflow:s wmem										
		⊕ sccpu_dataflow:s wn[4..0]		01h	X	04h	X	05h	X	1Fh	X	08h



单周期 CPU+ 指令存储器 + 数据存储器框图



- 顶层文件为 DE1_SOC_golden_top.v，该文件包含了输入输出参数，包括 10 个拨码开关 SW，4 个按键 KEY，6 个七段数码 HEX，10 个 LEDR。在工程中只用到 1 了复位键 KEY[0]，其余都没有使用。可供设计者作为输入输出使用 (可选)，如用 SW 控制输入，4 个 HEX 为输出，2 个 HEX 为操作码。
- DE1_SOC_golden_top.v 分层读，先了解主干，再了解枝干和具体实现细节。
- 从 DE1_SOC_golden_top.v 中可以看出，最重要的是实例化数据通路 sccpu_dataflow，于是应该看文件 sccpu_dataflow.v，该文件中倒数第二行有 alu 的例化，于是看 alu.v，打开后发现正是之前实验用到的 alu (a,b,aluc,r,z)，参数含义和实现方法都已经知道。



sccpu 代码导读

- 回到 `sccpu_dataflow.v`，可以看到控制器的例化：
`sccu_dataflow cu`，打开 `sccu_dataflow.v`
`sccu_dataflow` 的参数表如下：
(`op,func,z,wmem,wreg,regrt,m2reg,aluc,shift,al`
`uimm,pcsource,jal,sext`)
其中 **`op,func,z`** 为**输入**，其余为输出的**控制信号**。
- 观察代码发现信号的产生方式就是课上讲过的控制器的具体实现。实现时将与运算 `&` 和或运算 `|` 分步进行，先统一按照指令进行了 `&`，然后再 `|`。



sccpu 代码导读

- 再回到 `sccpu_dataflow.v`，可以看到寄存器堆 `regfile` 的例化：`regfile rf`，打开 `regfile.v`，其参数表如下：
(`rna`, `rnb`, `d`, `wn`, `we`, `clk`, `clrn`, `qa`, `qb`)
其中 **`qa`, `qb`** 为输出，其余为输入的控制信号，如 `rna`, `rnb` 分别为 `qa`, `qb` 的输出控制，`we` 为写入允许，`wn` 为选中的寄存器地址。
- 观察 `regfile.v` 可见寄存器堆有 32 个寄存器，每个 32 位。



sccpu 代码导读

- 仍然回到 `sccpu_dataflow.v`，可以看到多路开关的例化：
`mux2x32 alu_b (data,immediate,aluimm,alub);`
`mux2x32 alu_a (ra,sa,shift,alua);`
`mux2x32 result (alu,mem,m2reg,alu_mem);`
`mux2x32 link (alu_mem,p4,jal,res);`
`mux2x5 reg_wn (inst[15:11],inst[20:16],regrt,`
`reg_dest);`
`assign wn = reg_dest | {5{jal}};`
- 它们各自控制着不同支路，包括参加计算的数据来源，计算结果，是否写入寄存器；
- 下地址的产生：
`mux4x32 nextpc (p4,adr,ra,jpc,pcsource,npc);`
输出 `npc` 是 32 位的下地址，经 `dff32` 转为 `pc` 信号。



sccpu 代码导读

- 下地址的产生在 `sccpu_dataflow.v` 中：
`mux4x32 nextpc (p4,adr,ra,jpc,pcsource,npc);`
输出 `npc` 是 32 位的下地址，经 `dff32` 转为 `pc` 信号，4 条支路代表产生下地址的 4 种方式，如下：
`pcplus4` 用 `cla32` 实现 `pc+4`，即 `p4`；
`br_adr` 用 `cla32` 实现 `p4+offset`，即 `adr`；
`ra` 是从寄存器中读出来的，见 `regfile`；
`jpc={p4[31:28],inst[25:0],2 'b00}`，是指令和 `pc` 拼出来的；



sccpu 代码导读

- 时钟在顶层文件中
把外部的 **CLOCK_50** 用 PLL 转换成 **clk_10MHz** 给工程使用。
- 存储器的使用在顶层文件中：
scinstmem imem (pc,inst);
sccdatamem dmem (clk_10MHz, memout, data, aluout,
wmem, clk_10MHz, clk_10MHz);



实验要求

- 基本要求：工程中已经包含了源文件，请同学们理清相互关系。看懂 Verilog 代码，并用 signaltap 证实运行正确；
- 可选要求：若能有所改进提高，就好得不能再好了。包括：扩展其他指令，用 LED 呈现程序的运行结果等。
- 先做最基础的，要看懂，尤其是模块间的相互关系。
- 为便于讨论和分工合作，CPU 实验项目采取分组，2 人一组。最后一次课进行答辩演示。
- 每组同学现场演示实验结果，并采用 ppt 进行汇报 + 答辩。结束后，每位同学提交一份实验报告（模板）。以小组为单位提交 ppt+ 项目文档。
- 答辩现场老师随机提问，每个人都要回答问题。



附录：CPU 支持的 20 条指令

指令	[31:26]	[25:21]	[20:16]	[15:11]	[10:6]	[5:0]	意 义
add	000000	rs	rt	rd	00000	100000	寄存器加
sub	000000	rs	rt	rd	00000	100010	寄存器减
and	000000	rs	rt	rd	00000	100100	寄存器与
or	000000	rs	rt	rd	00000	100101	寄存器或
xor	000000	rs	rt	rd	00000	100110	寄存器异或
sll	000000	00000	rt	rd	sa	000000	左移
srl	000000	00000	rt	rd	sa	000010	逻辑右移
sra	000000	00000	rt	rd	sa	000011	算术右移
jr	000000	rs	00000	00000	00000	001000	寄存器跳转
addi	001000	rs	rt	immediate			立即数加
andi	001100	rs	rt	immediate			立即数与
ori	001101	rs	rt	immediate			立即数或
xori	001110	rs	rt	immediate			立即数异或
lw	100011	rs	rt	offset			取整数数据字
sw	101011	rs	rt	offset			存整数数据字
beq	000100	rs	rt	offset			相等转移
bne	000101	rs	rt	offset			不等转移
lui	001111	00000	rt	immediate			设置高位
j	000010	address					跳转
jal	000011	address					调用



附录：指令对应的控制信号，控制器就是根据这个设计的。

指令	z	wreg	regrt	jal	m2reg	shift	aluimm	sext	aluc[3:0]	wmem	pcsource[1:0]
add	x	1	0	0	0	0	0	x	x 0 0 0	0	0 0
sub	x	1	0	0	0	0	0	x	x 1 0 0	0	0 0
and	x	1	0	0	0	0	0	x	x 0 0 1	0	0 0
or	x	1	0	0	0	0	0	x	x 1 0 1	0	0 0
xor	x	1	0	0	0	0	0	x	x 0 1 0	0	0 0
sll	x	1	0	0	0	1	0	x	0 0 1 1	0	0 0
srl	x	1	0	0	0	1	0	x	0 1 1 1	0	0 0
sra	x	1	0	0	0	1	0	x	1 1 1 1	0	0 0
jr	x	0	x	x	x	x	x	x	x x x x	0	1 0
addi	x	1	1	0	0	0	1	1	x 0 0 0	0	0 0
andi	x	1	1	0	0	0	1	0	x 0 0 1	0	0 0
ori	x	1	1	0	0	0	1	0	x 1 0 1	0	0 0
xori	x	1	1	0	0	0	1	0	x 0 1 0	0	0 0
lw	x	1	1	0	1	0	1	1	x 0 0 0	0	0 0
sw	x	0	x	x	x	0	1	1	x 0 0 0	1	0 0
beq	0	0	x	x	x	0	0	1	x 0 1 0	0	0 0
beq	1	0	x	x	x	0	0	1	x 0 1 0	0	0 1
bne	0	0	x	x	x	0	0	1	x 0 1 0	0	0 1
bne	1	0	x	x	x	0	0	1	x 0 1 0	0	0 0
lui	x	1	1	0	0	x	1	x	x 1 1 0	0	0 0
j	x	0	x	x	x	x	x	x	x x x x	0	1 1
jal	x	1	x	1	x	x	x	x	x x x x	0	1 1



附录：控制信号的功能说明

控制信号	意 义
wreg	写寄存器：为 1 时写寄存器；否则不写
regrt	目的寄存器号是 rt：为 1 时选择 rt；否则选择 rd
jal	子程序调用：为 1 时表示指令是 jal；否则不是
m2reg	存储器数据写入寄存器：为 1 时选择存储器数据；否则选择 ALU 结果
shift	ALU a 使用移位位数：为 1 时使用移位位数；否则使用寄存器数据
aluimm	ALU b 使用立即数：为 1 时使用立即数；否则使用寄存器数据
sext	立即数符号扩展：为 1 时符号扩展；否则零扩展
aluc[3:0]	ALU 操作控制：见第 4 章图 4.6
wmem	写存储器：为 1 时写存储器；否则不写
pcsource[1:0]	下一条指令地址的选择：00 选 PC + 4；01 选转移地址； 10 选寄存器内的地址；11 选跳转地址



附录：rom 中的程序流程

```
assign rom[5'h00] = 32'h3c010000; // (00) main: lui r1,0
assign rom[5'h01] = 32'h34240050; // (04)         ori r4,r1,80
assign rom[5'h02] = 32'h20050004; // (08)         addi r5,r0, 4
assign rom[5'h03] = 32'h0c000018; // (0c) call: jal sum
assign rom[5'h04] = 32'hac820000; // (10)         sw r2,0(r4)
assign rom[5'h05] = 32'h8c890000; // (14)         lw r9, 0(r4)
assign rom[5'h06] = 32'h01244022; // (18)         sub r8, r9, r4
assign rom[5'h07] = 32'h20050003; // (1c)         addi r5, r0, 3
assign rom[5'h08] = 32'h20a5ffff; // (20) loop2: addi r5, r5, -1
assign rom[5'h09] = 32'h34a8ffff; // (24)         ori r8, r5, 0xffff
assign rom[5'h0A] = 32'h39085555; // (28)         xori r8, r8, 0x5555
assign rom[5'h0B] = 32'h2009ffff; // (2c)         addi r9, r0, -1
assign rom[5'h0C] = 32'h312affff; // (30)         andi r10, r9, 0xffff
assign rom[5'h0D] = 32'h01493025; // (34)         or r6, r10, r9
assign rom[5'h0E] = 32'h01494026; // (38)         xor r8, r10, r9
assign rom[5'h0F] = 32'h01463824; // (3c)         and r7, r10, r6
assign rom[5'h10] = 32'h10a00001; // (40)         beq r5, r0, shift
assign rom[5'h11] = 32'h08000008; // (44)         j loop2
assign rom[5'h12] = 32'h2005ffff; // (48) shift: addi r5, r0, -1
assign rom[5'h13] = 32'h000543c0; // (4c)         sll r8, r5, 15
assign rom[5'h14] = 32'h00084400; // (50)         sll r8, r8, 16
assign rom[5'h15] = 32'h00084403; // (54)         sta r8, r8, 16
assign rom[5'h16] = 32'h000843c2; // (58)         srl r8, r8, 15
assign rom[5'h17] = 32'h08000017; // (5c) finish: j finish
assign rom[5'h18] = 32'h00004020; // (60) sum: add r8, r0, r0
assign rom[5'h19] = 32'h8c890000; // (64) loop: lw r9, (r4)
assign rom[5'h1A] = 32'h20840004; // (68)         addi r4, r4, 4
assign rom[5'h1B] = 32'h01094020; // (6c)         add r8, r8, r9
assign rom[5'h1C] = 32'h20a5ffff; // (70)         addi r5, r5, -1
assign rom[5'h1D] = 32'h14a0ffff; // (74)         bne r5, r0, loop
assign rom[5'h1E] = 32'h00081000; // (78)         sll r2, r8, 0
assign rom[5'h1F] = 32'h03e00008; // (7c)         jr r31
assign inst = rom[a[6:2]];
```

跳转

循环

程序以死
循环结尾

返回



附录：alu 支持的 9 种运算

20 条指令中共涉及 9 种 alu 的运算，即 alu 的控制信号 aluc 有 9 个取值。

aluc[3:0]

x000 ADD

x100 SUB

x001 AND

x101 OR

x010 XOR

x110 LUI

0011 SLL

0111 SRL

1111 SRA



附录：拨码开关和七段数码管标号和引脚分配
(如使用 DE1_SOC.qsf 文件，其中提供了所有引脚的分配，标号在一的右侧)

# SW	# SEG7	
PIN_AB12 -- SW[0]	PIN_AE26 -- HEX0[0]	PIN_AB23 -- HEX2[0]
PIN_AC12 -- SW[1]	PIN_AE27 -- HEX0[1]	PIN_AE29 -- HEX2[1]
PIN_AF9 -- SW[2]	PIN_AE28 -- HEX0[2]	PIN_AD29 -- HEX2[2]
PIN_AF10 -- SW[3]	PIN_AG27 -- HEX0[3]	PIN_AC28 -- HEX2[3]
PIN_AD11 -- SW[4]	PIN_AF28 -- HEX0[4]	PIN_AD30 -- HEX2[4]
PIN_AD12 -- SW[5]	PIN_AG28 -- HEX0[5]	PIN_AC29 -- HEX2[5]
PIN_AE11 -- SW[6]	PIN_AH28 -- HEX0[6]	PIN_AC30 -- HEX2[6]
PIN_AC9 -- SW[7]	PIN_AJ29 -- HEX1[0]	PIN_AD26 -- HEX3[0]
PIN_AD10 -- SW[8]	PIN_AH29 -- HEX1[1]	PIN_AC27 -- HEX3[1]
PIN_AE12 -- SW[9]	PIN_AH30 -- HEX1[2]	PIN_AD25 -- HEX3[2]
	PIN_AG30 -- HEX1[3]	PIN_AC25 -- HEX3[3]
	PIN_AF29 -- HEX1[4]	PIN_AB28 -- HEX3[4]
	PIN_AF30 -- HEX1[5]	PIN_AB25 -- HEX3[5]
	PIN_AD27 -- HEX1[6]	PIN_AB22 -- HEX3[6]



附录：QuartusII 备忘

- 工程的设置在 assignments->setting 下，如增减文件、signal tap 的开关、Verilog 版本等。
- 设置 top_level 在左侧 project navigator 的 Files 页选中后右键 set as top_level entity。
- Assignments 下的 pin planner 分配引脚，Device 选择器件。
- 烧写步骤：在 Tools->programmer 界面下：
确保 Hardware setup 是正确的，应为 DE-SOC[USB-1]
点击 Auto Detect，选 5CSEMA5 OK 出现的界面，选中 5CSEMA5 行 (SOCVHPS 不管)，点击 Change file (不是 Add file) 然后选择要写入的 *.sof 文件，将 sof 文件调入后勾选 program/configure, 最后点击 start 按钮开始写入。*.sof 掉电丢失，*.pof 掉电不丢失。
- 如果 QuartusII 的 programmer 未能自动检测到 DE-SOC，则需要先在 Hardware setup 窗口下的 Currently selected hardware 选择 DE-SOC[USB-1], 然后 Close。



附录：SignalTap II 的使用

Signal Tap II

by 计科 1603 彭博洋



1. **SignalTap** : 随着 **FPGA** 容量的增大, 设计的复杂, 设计调试任务繁重; **SignalTap** 作为一种简易有效的调试工具应运而生, 旨在缩短测试时间。它具有无干扰, 便于升级, 使用简单, 价格低廉等优点。
2. 使用 **SignalTap** 产生的文件, 其后缀名为 **.stp**, 与工程中的其他文件处于同一个目录下

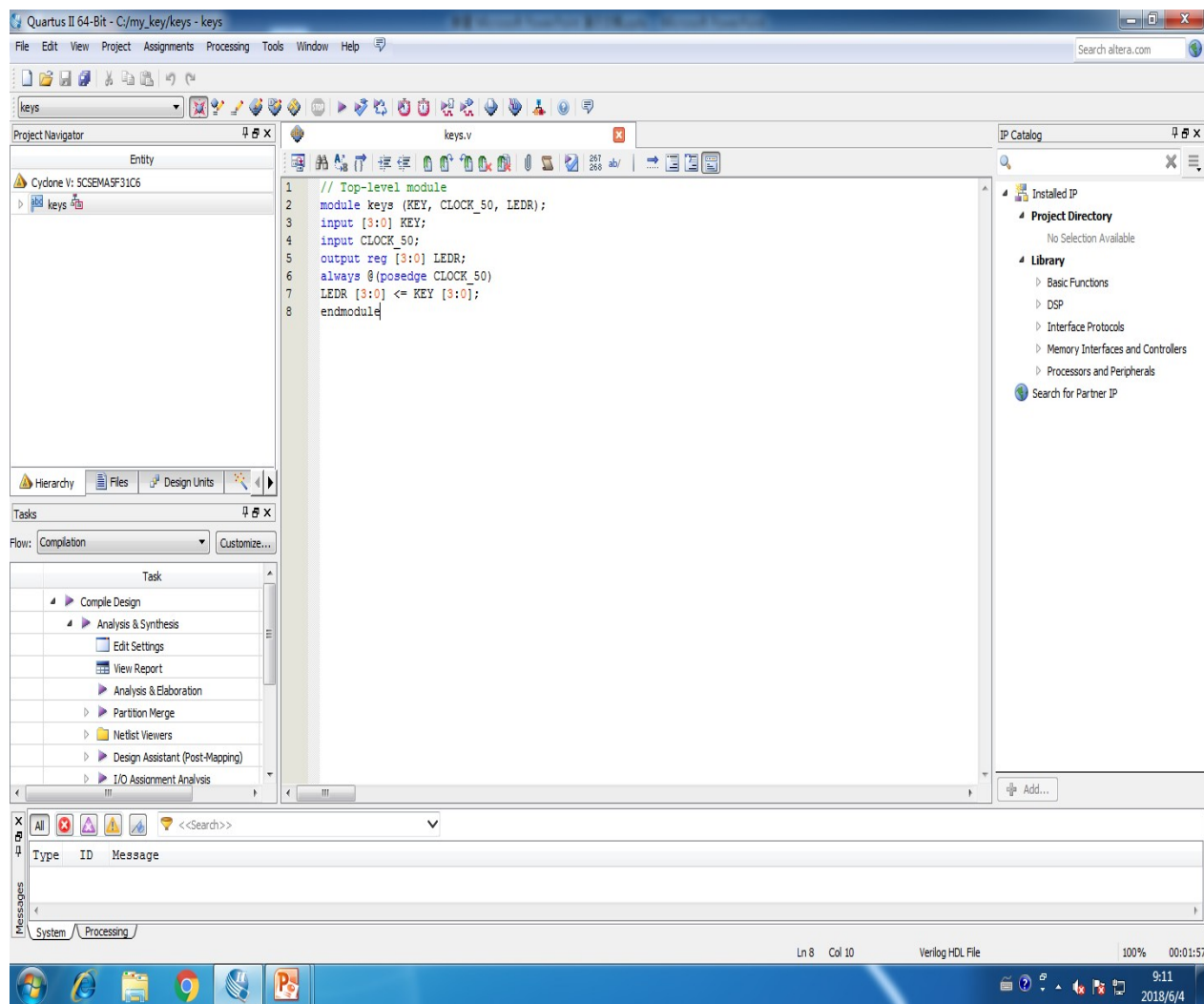


区别：

1. 与传统的仿真工具（如 **Modelsim**）相比：每次都要综合整个工程，再下载代码，然后才可以使用 **SignalTap**，这说明了 **SignalTap** 是由一些逻辑电路组成，而不是仿真。并且 **SignalTap** 界面显示的波形为实际的波形，而仿真工具界面显示的波形为仿真的波形。即，我们在使用 **SignalTap** 时，是对开发板进行操作，而不是软件中的虚拟数据。
2. 与直接下载到开发板相比：直接下载到开发板如果不正确，是无法得知哪个版块出现了异常，只能重头再来，像个无头苍蝇，凭感觉去判断。而通过 **SignalTap** 这个调试工具，可以全程跟踪变量，通过界面上显示的波形和实际开发板上进行比对，可以很容易得发现程序的问题所在。



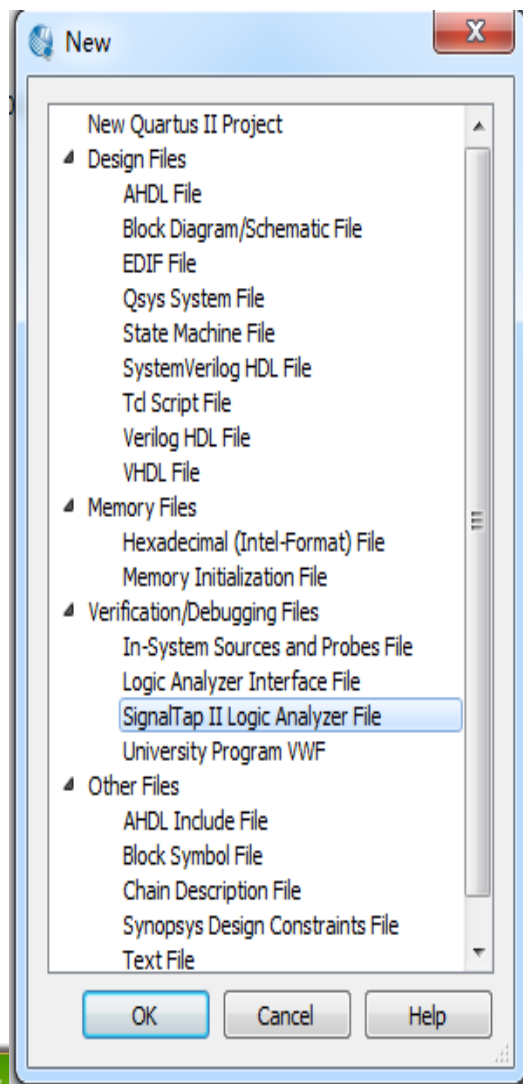
流程：（以第一次实验的 my_key 为例）



打开工程文件夹



new→file →SignalTap II Analyzer File→OK





SignalTap II Logic Analyzer - E:/jzsy/DE1_SOC_test_sy3/DE1_SOC_test_sy3/DE1_SOC_golden_top - DE1_SOC_golden_top - [stp1.stp]*

File Edit View Project Processing Tools Window Help

Search altera.com

Instance Manager: Add nodes to the current instance

Instance	Status	Enabled	LEs: 0	Memory: 0	Small: NA	Medium: NA	Large: NA
auto_signaltap_0	Not running	☒	0 cells	0 bits	NA	NA	NA

JTAG Chain Configuration: JTAG ready

Hardware: DE-SoC [USB-1] Setup...

Device: @2: 5CSE(BA5)MA5)5CSTFD5D Scan Chain

>> SOF Manager: 开发板配置栏

auto_signaltap_0 Lock mode: Allow all changes

Node		Data Enable	Trigger Enable	Trigger Conditions	
Type	Alias	Name	0	0	1 Basic AND
Double-click to add nodes					

节点栏

Signal Configuration:

Clock:

Data

Sample depth: 128 RAM type: Auto

☐ Segmented: 2 64 sample segments

Nodes Allocated: ☒ Auto ☐ Manual: 0

Storage qualifier:

Type: Continuous

Input port:

Nodes Allocated: ☒ Auto ☐ Manual: 0

☒ Record data discontinuities

☐ Disable storage qualifier

!!!

Signal Configuration: 信号配置栏

Hierarchy Display:

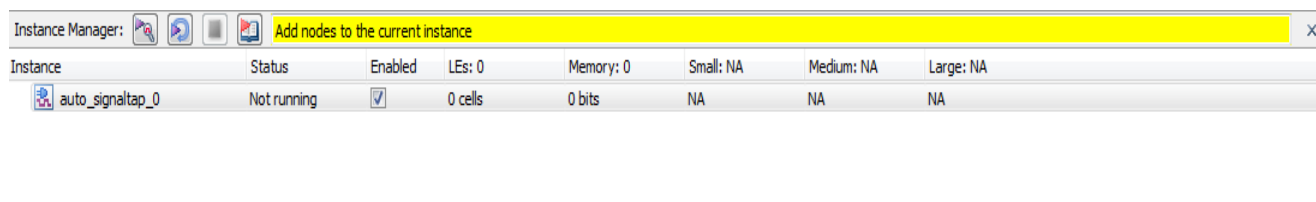
Data Log: auto_signaltap_0

auto_signaltap_0

0% 00:00:00



实例栏和提示框：



提示框起引导反馈作用，一般通过提示框得知下一步操作和操作失败问题所在

实例栏中可以声明实例化对象，然后通过给对象添加节点，实现对节点的追踪调试，不同的对象可以观测不同的节点，也可以通过一个对象进行全员观测。



双击空白区域进行节点的添加（黄色提示框会进行步骤引导）

Instance Manager:     Add nodes to the current instance

Instance	Status	Enabled	LEs: 0	Memory: 0	Small: NA	Medium: NA	Large: NA
 auto_signaltap_0	Not running	☒	0 cells	0 bits	NA	NA	NA

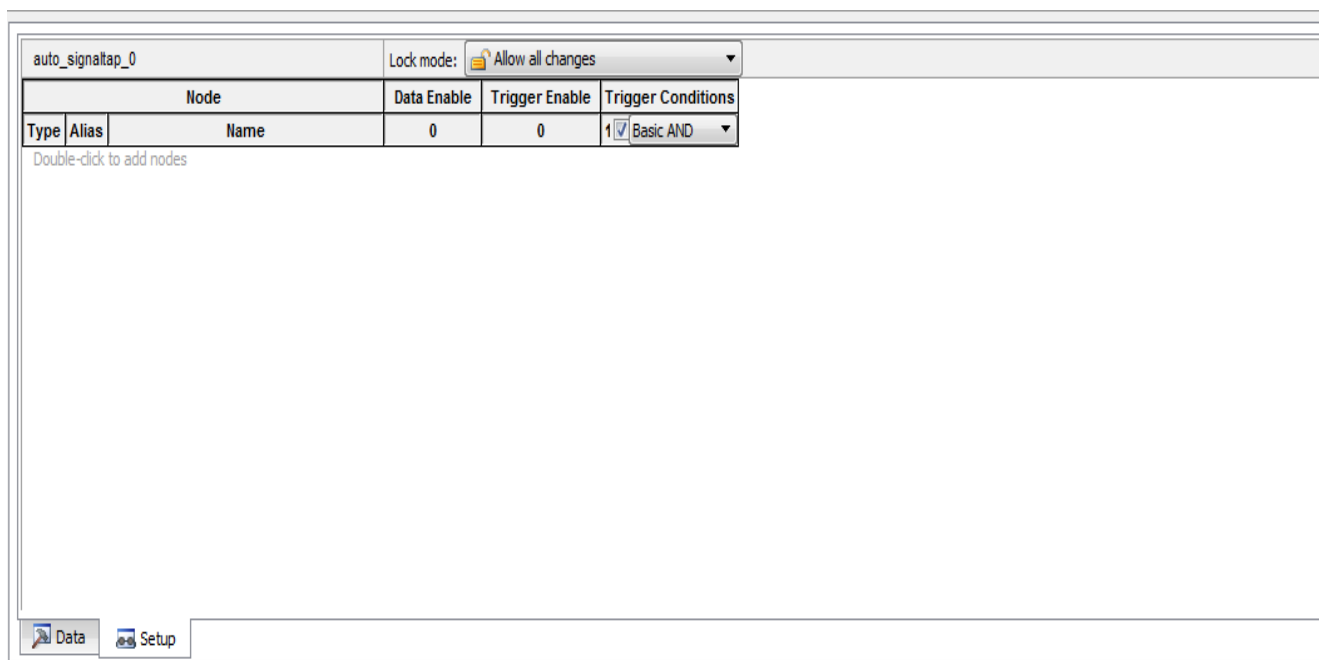
auto_signaltap_0 Lock mode:  Allow all changes

Node			Data Enable	Trigger Enable	Trigger Conditions
Type	Alias	Name	0	0	1 ☒ Basic AND
Double-click to add nodes					

Data Setup



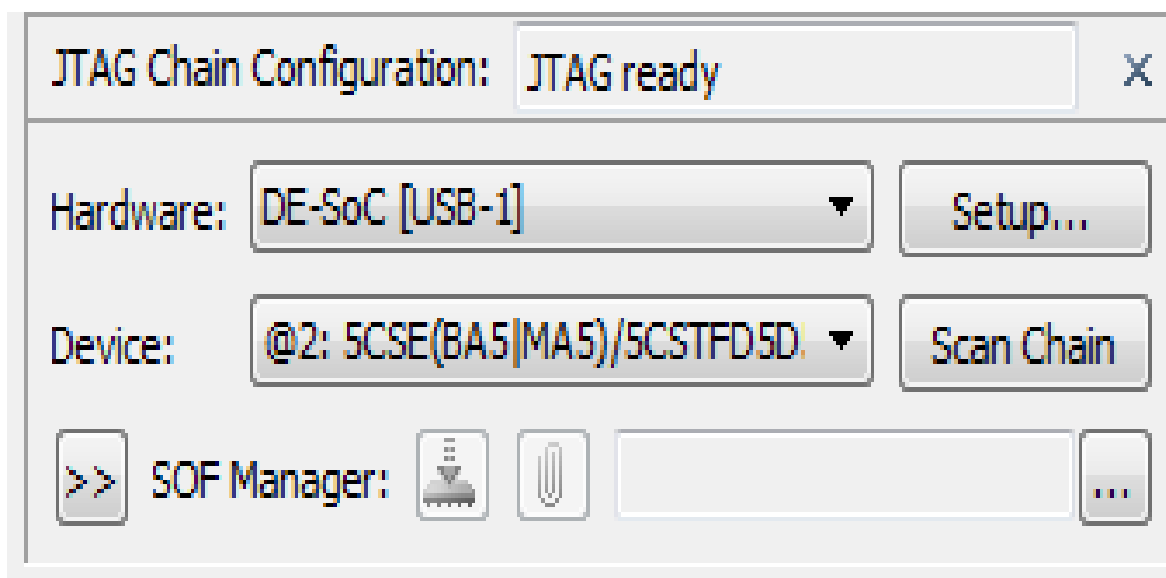
节点栏：



可以观察到节点的类型，别名，名字，数据使能，触发使能（拨动开关或按下按钮等）和触发器的状态（Basic AND的意思是：只有下面的所有触发条件都成立，SignalTap II才开始工作。trigger conditions的意思是：比如说一个信号被设置成下降沿出发，则只有在这个信号的下降沿开始SignalTapII才开始采集数据）



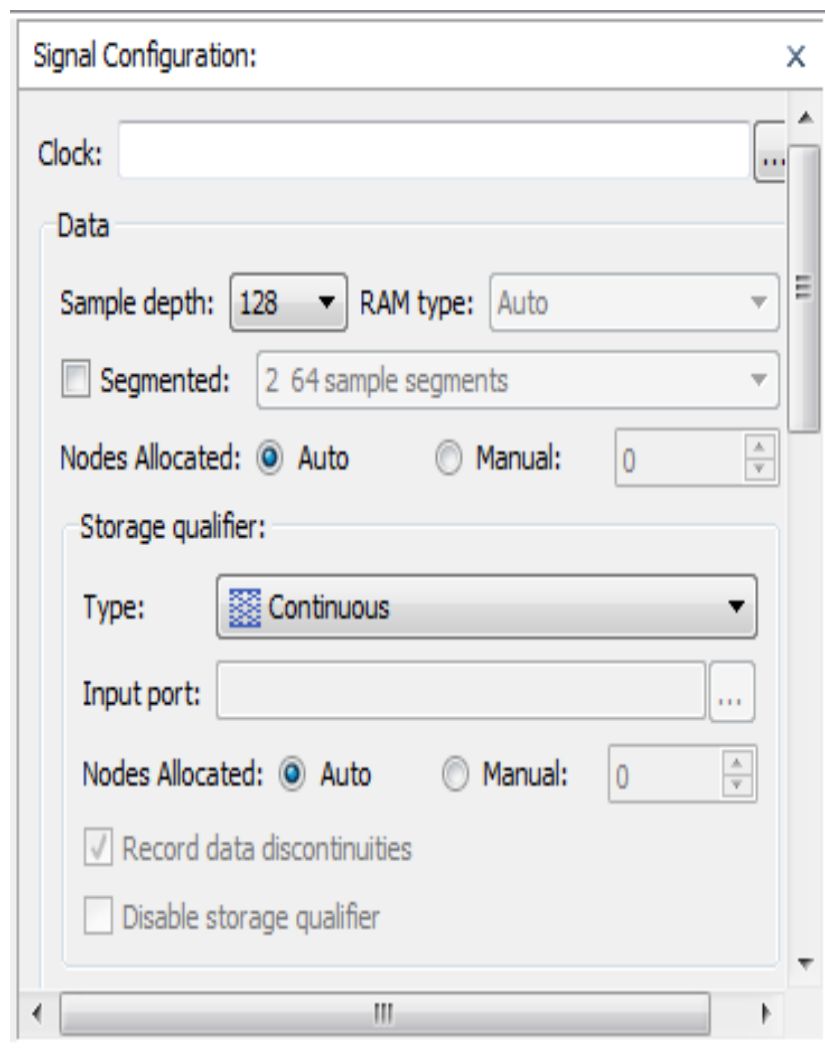
开发板配置栏：



硬件选择当然是 DE-SoC 开发板，Device 选择 @2 。第三栏的 SOF 文件后续进行选择。



信号配置栏：

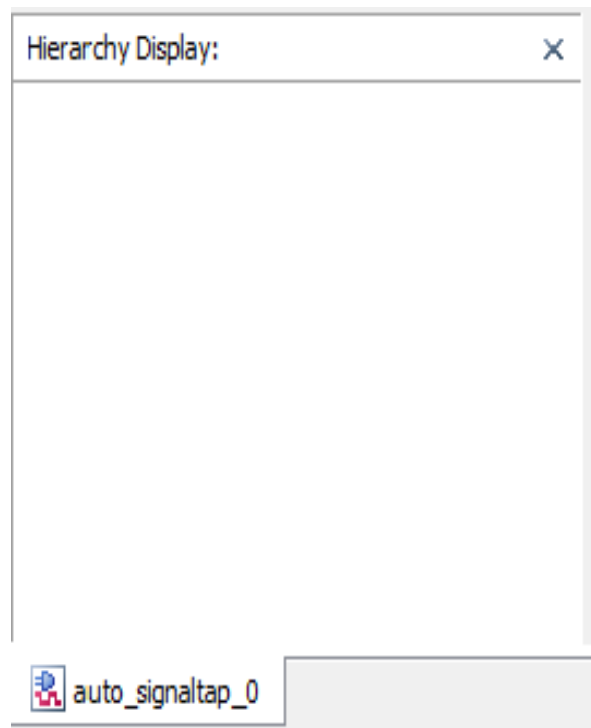


Clock：采样信号，一般选择 CLOCK_50

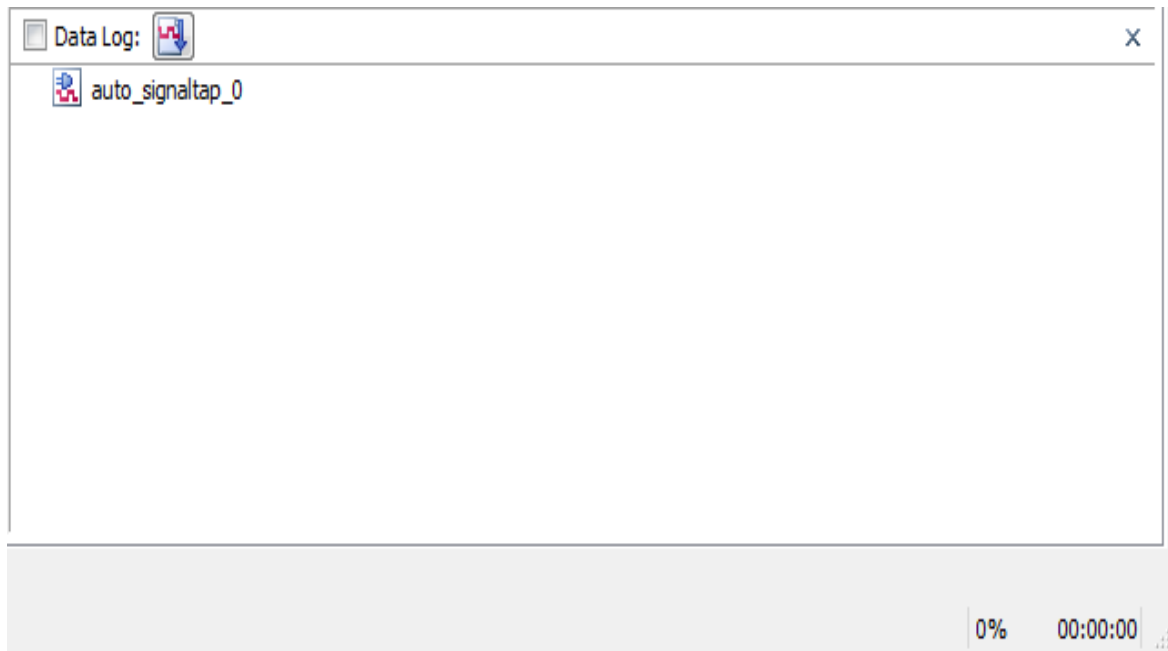
Sample depth：采样深度，也就是采多少个点，这也是看具体情况而定，深度越大，需要的资源越多。不过深度过大时，资源分配不是会报错。暂时不需要用到，在此不做介绍。如果往下拖动过多导致界面跳转，可以按 `ctrl+z` 进行回滚。（快捷键在该界面均可使用）



其他版块：



层次显示：目前只用到了一层，因此不需要太过于关注。



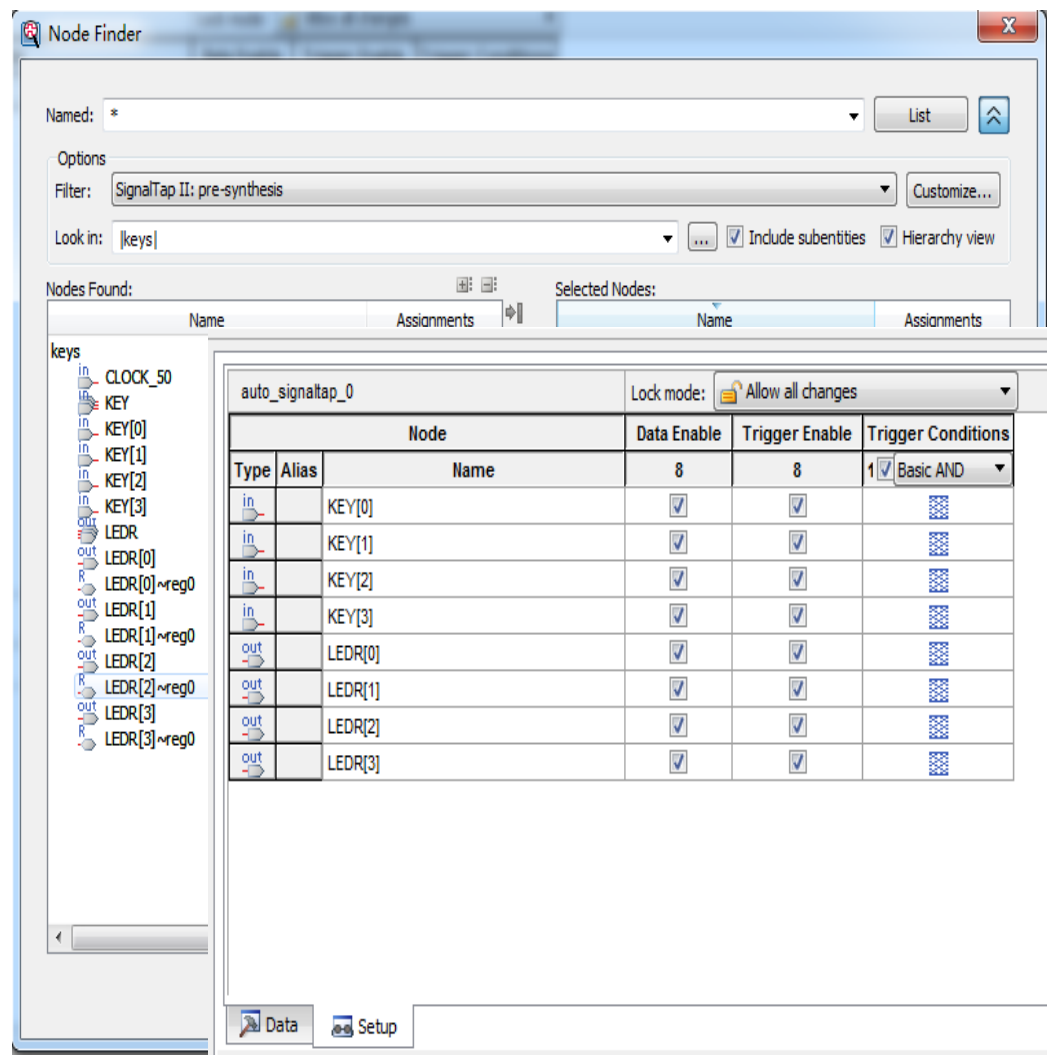
数据日记：记录了操作过程和其他杂七杂八的内容。



1. 添加节点:

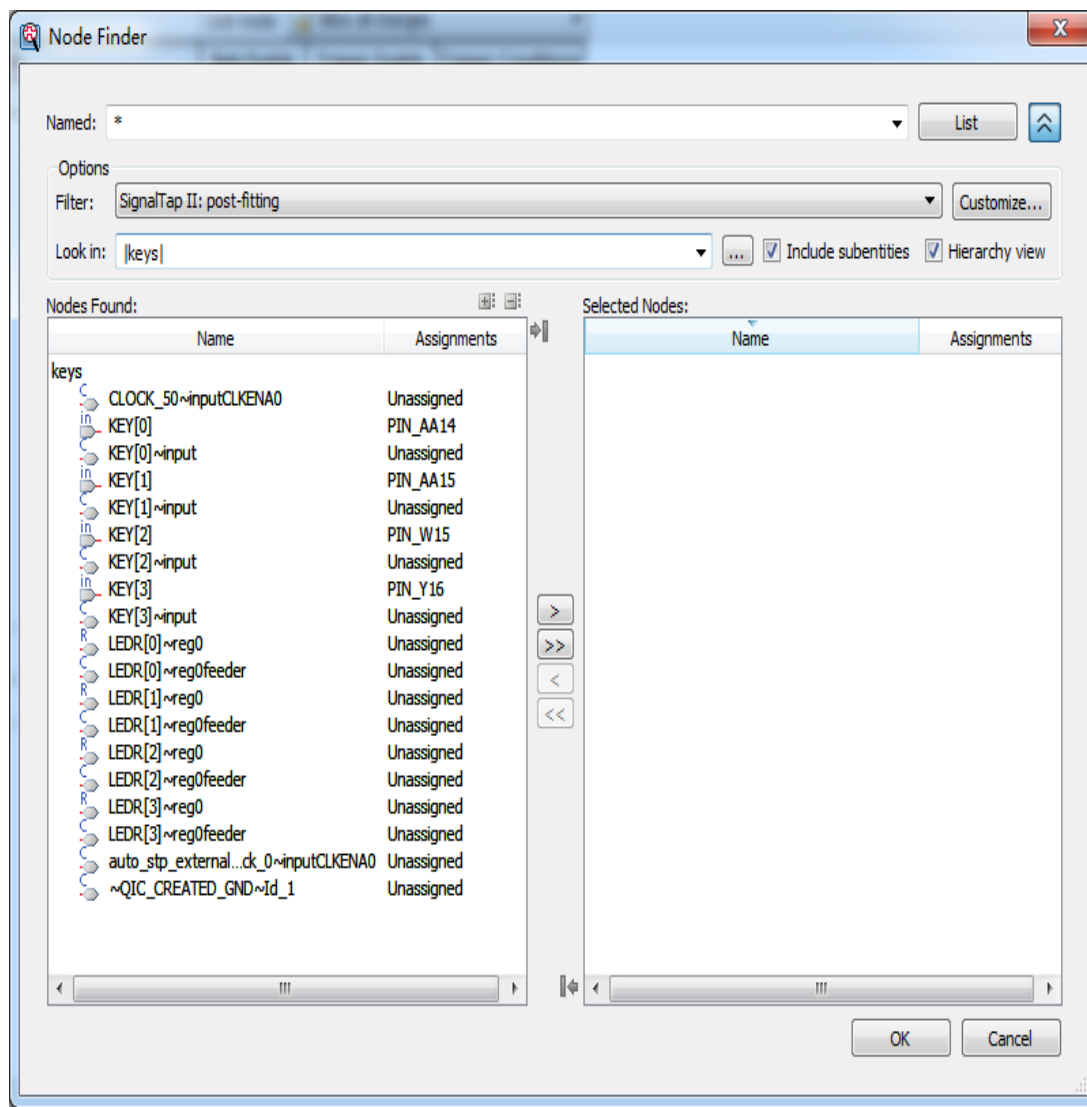
Filter 选择 pre-synthesis，按下 List，节点选择后移入右边，按下 ok 即选择完毕。

关于 pre-synthesis 和 post-fitting，一般：有“预谋”的使用 signaltap 就使用 pre-synthesis，“意外”情况就使用 post-fitting





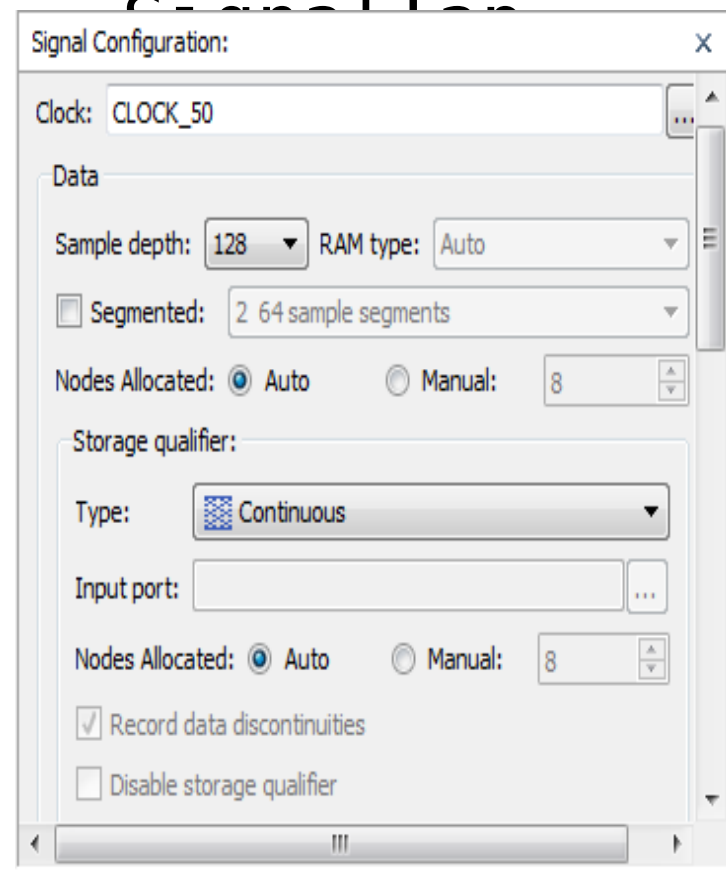
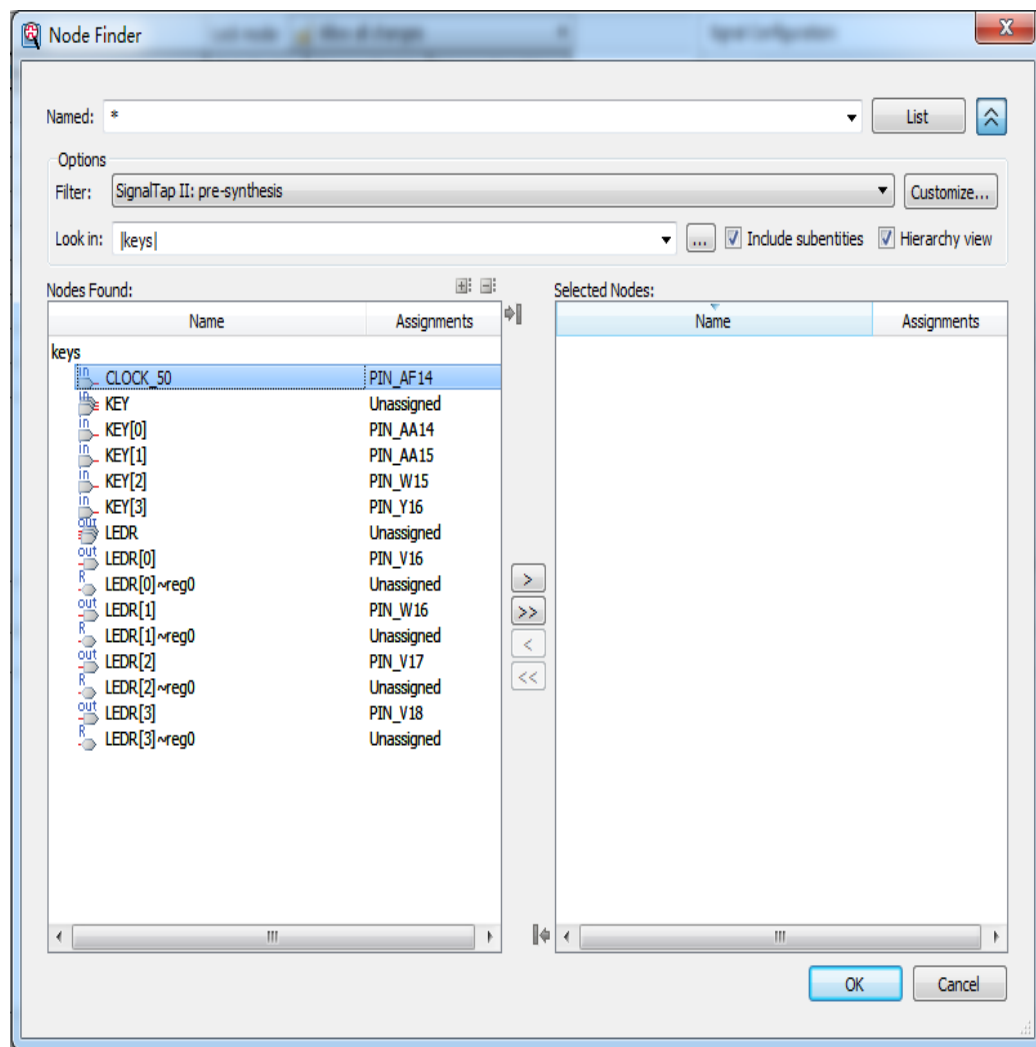
post-fitting 下的节点: (可自行了解)





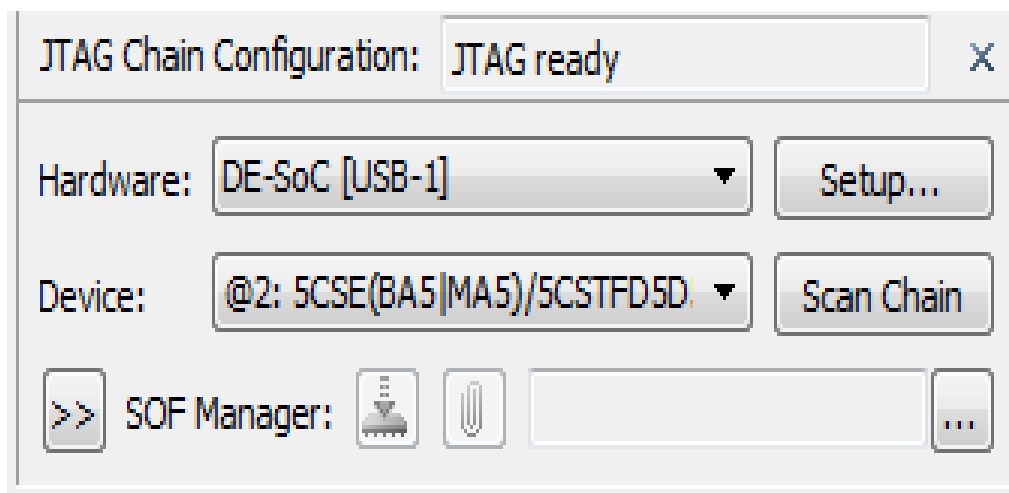
2. 信号设置：同样选择 pre-synthesis (前缀是

SignalTap

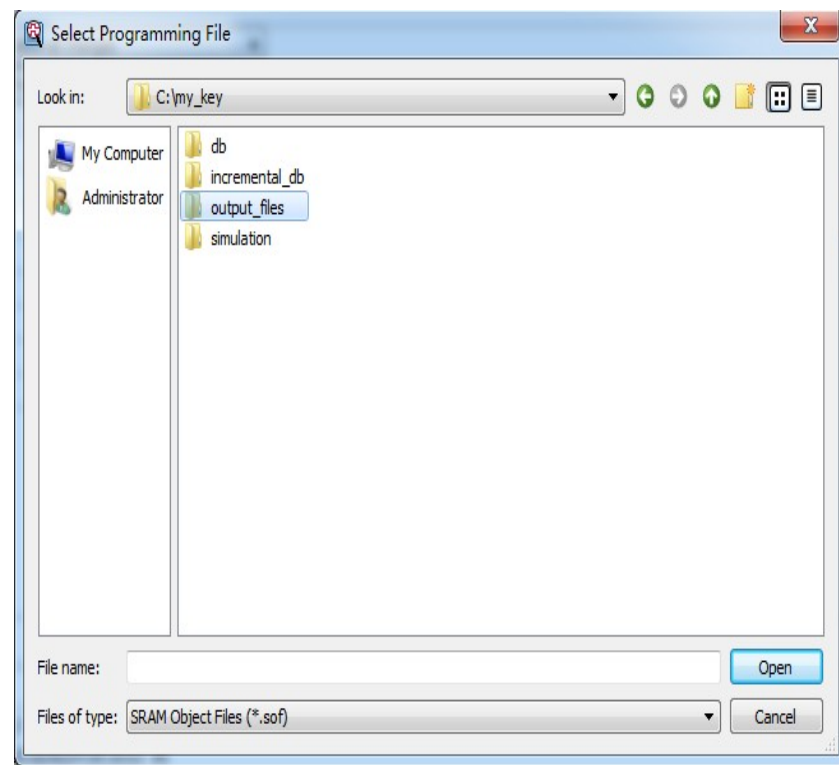




3. 硬件设置（已经默认设置了一部分）



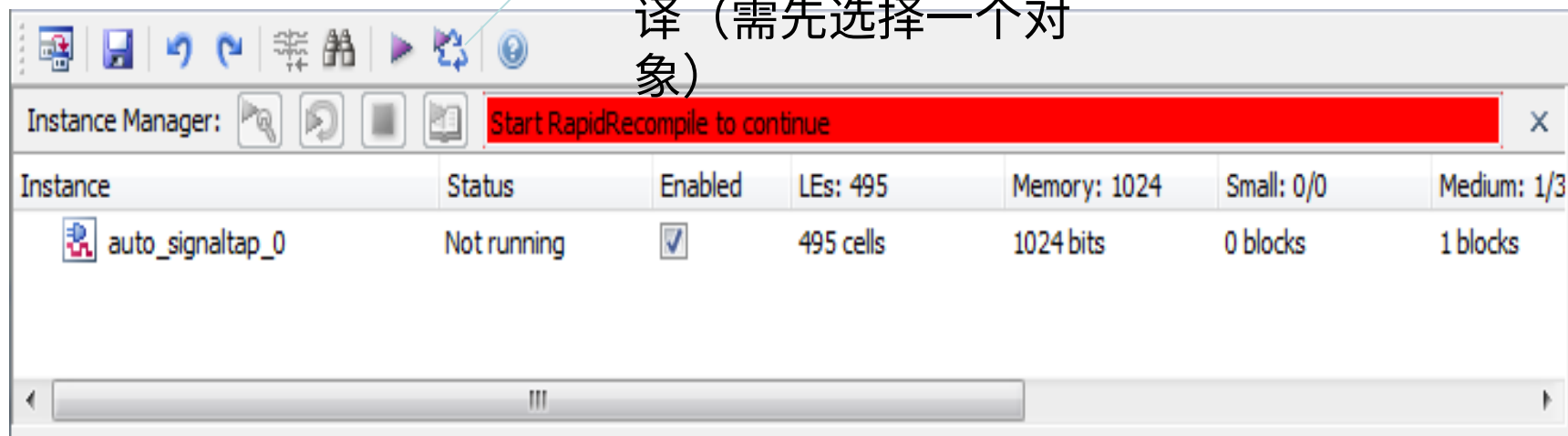
点击右下角的搜索按钮，
在 output_files 文
件夹下找到 .sof 文件，
进行添加





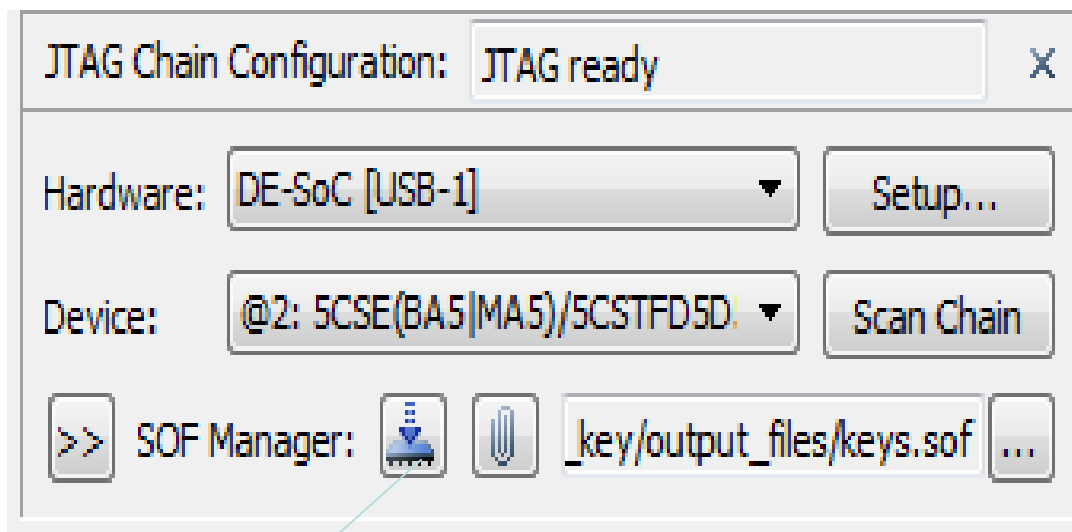
4. 编译

编译按钮，也可回到
quartus 页面进行编
译（需先选择一个对
象）

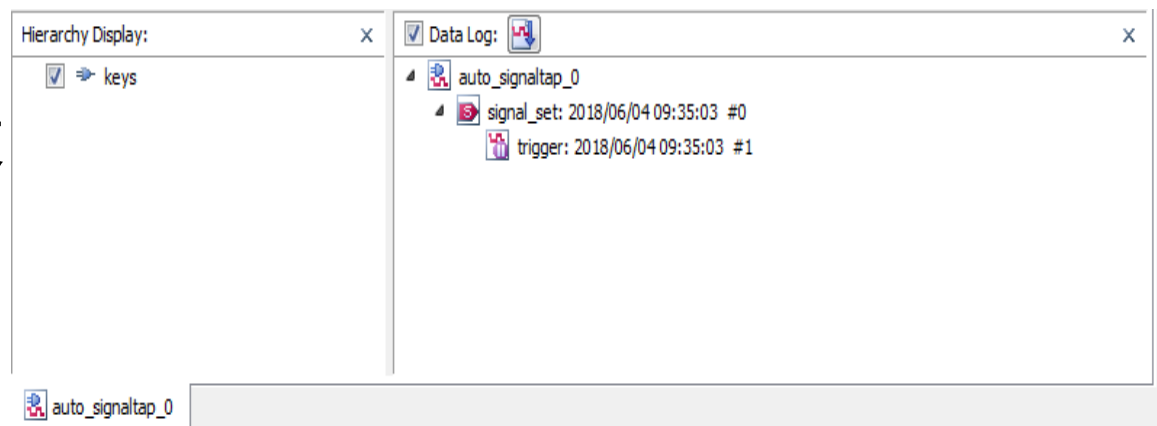




5. 下载

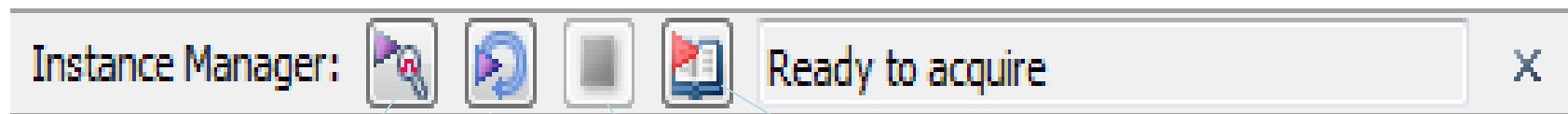


点击下载到开发板





6. 运行（建议循环运行，比较省事）



单次运行

循环运行
(不按停止
则不会
停止)

停止运行
(不停止
则无法关
闭界面)

转到数据界面
(自动跳转)



运行界面：

拨动开关，则对应的波形会发生变化，从而进行数据的追踪

按一下

name 的右边竖线，则会显示值，比波形更易观测！也可按自己喜好调成其他进制

log: Trig @ 2018/06/04 09:46:00 (0:0:0.2 elapsed) #2

Type	Alias	Name	-62 Value -61
		KEY[0]	1
		KEY[1]	1
		KEY[2]	1
		KEY[3]	1
		LEDR[0]	1
		LEDR[1]	1
		LEDR[2]	1
		LEDR[3]	1



谢谢观看，如有错误，欢迎探讨分享。